

A PROJECT REPORT

ON

“Mini PDF Processing Website”



Submitted By :- Nitesh Kumar Sharma
Email id:-niteshsharmaaurangabad1234@gmail.com
Contact No:-7633968978
Company Name:- Techinterex Gurugram Haryana.
HR :- Divya Rajput
Job type:- Internship (Six months)
Starting Date:-01-11-2024
Ending Date:- 31-04-2025

OVERVIEW OF PROJECT

- Project Features (MVP - Minimum Viable Product)
 - 1. Upload PDF: Users can upload one or more PDF files.
 - 2. Merge PDFs: Combine multiple PDFs into one.
 - 3. Split PDFs: Extract specific pages from a PDF.
 - 4. Compress PDFs: Reduce the file size of a PDF.
 - 5. Download Processed PDF: Provide a download link for the processed PDF.
 - 6. Word to PDF: Convert DOC and DOCX files into PDFs.
 - 7. PDF to Word: Convert PDFs back into editable DOC and DOCX files.
 - 8. JPG to PDF: Convert JPG images into PDF.
 - 9. PDF to JPG: Convert PDF pages into JPG images.
 - 10. Simple UI: Provide a user-friendly interface with drag-and-drop file upload.

FRONTEND

- `<!DOCTYPE html>`
- `<html>`
- `<head>`
- `<title>Mini PDF App</title>`
- `<link rel="stylesheet" href="style.css">`
- `</head>`
- `<body>`
- `<h1>Mini PDF App</h1>`
- `<input type="file" id="pdfInput" multiple />`
- `<button onclick="upload()">Upload</button>`
- `<button onclick="merge()">Merge PDFs</button>`
- `<button onclick="split()">Split PDF (first page)</button>`
- `<script src="script.js"></script>`
- `</body>`
- `</html>`

Next

```
• <!DOCTYPE html>
• <html lang="en">
• <head>
•   <meta charset="UTF-8">
•   <title>PDF Processor</title>
•   <style>
•     #drop-area {
•       border: 2px dashed #ccc;
•       padding: 20px;
•       text-align: center;
•       margin: 20px;
•     }
•   </style>
• </head>
• <body>
•   <h2>Upload PDF Files</h2>
•   <div id="drop-area">
•     <p>Drag & drop files here</p>
•     <input type="file" id="fileElem" multiple>
•     <button onclick="upload()">Upload</button>
•   </div>
•
•   <script>
•     function upload() {
•       const input = document.getElementById('fileElem');
•       const files = input.files;
•       const formData = new FormData();
•
•       for (let i = 0; i < files.length; i++) {
•         formData.append("files", files[i]);
•       }
•
•       fetch('/api/files/upload', {
•         method: 'POST',
•         body: formData
•       })
•       .then(res => res.text())
•       .then(data => alert(data))
•       .catch(err => alert("Error: " + err));
•     }
•   </script>
• </body>
• </html>
```

Upload PDF

- `@PostMapping("/upload")`
- `public ResponseEntity<String>`
`uploadFile(@RequestParam("file") MultipartFile file)`
`throws IOException {`
- `Path path = Paths.get("uploads/" +`
`file.getOriginalFilename());`
- `Files.createDirectories(path.getParent());`
- `Files.write(path, file.getBytes());`
- `return ResponseEntity.ok("File uploaded: " +`
`path.toString());`
- `}`

Merge PDF

- `@PostMapping("/merge")`
- `public ResponseEntity<String> mergePdfs(@RequestParam("files") MultipartFile[] files) throws IOException {`
- `PDFMergerUtility merger = new PDFMergerUtility();`
- `String outputPath = "uploads/merged.pdf";`
- `for (MultipartFile file : files) {`
- `Path path = Paths.get("uploads/" + file.getOriginalFilename());`
- `Files.write(path, file.getBytes());`
- `merger.addSource(path.toFile());`
- `}`
- `merger.setDestinationFileName(outputPath);`
- `merger.mergeDocuments(null);`
- `return ResponseEntity.ok("Merged PDF available at: " + outputPath);`
- `}`

Split PDF

- `@PostMapping("/split")`
- `public ResponseEntity<String> splitPdf(@RequestParam("file") MultipartFile file,`
- `@RequestParam("start") int start,`
- `@RequestParam("end") int end) throws IOException {`
- `PDDocument document = PDDocument.load(file.getInputStream());`
- `PDDocument split = new PDDocument();`
- `for (int i = start - 1; i < end; i++) {`
- `split.addPage(document.getPage(i));`
- `}`
- `String outputPath = "uploads/split.pdf";`
- `split.save(outputPath);`
- `split.close();`
- `document.close();`
- `return ResponseEntity.ok("Split PDF saved at: " + outputPath);`
- `}`

Compress PDF

- `@PostMapping("/compress")`
- `public ResponseEntity<String> compressPdf(@RequestParam("file") MultipartFile file) throws IOException {`
- `PDDocument document = PDDocument.load(file.getInputStream());`
- `for (PDPage page : document.getPages()) {`
- `PDResources res = page.getResources();`
- `for (COSName xObjectName : res.getXObjectNames()) {`
- `PDXObject xObject = res.getXObject(xObjectName);`
- `if (xObject instanceof PDImageXObject image) {`
- `image.getImage().flush(); // basic compression`
- `}`
- `}`
- `}`
- `String outputPath = "uploads/compressed.pdf";`
- `document.save(outputPath);`
- `document.close();`
- `return ResponseEntity.ok("Compressed PDF saved at: " + outputPath);`
- `}`

Download Pdf

- `@GetMapping("/download/{filename}")`
- `public ResponseEntity<Resource> downloadFile(@PathVariable String filename) throws IOException {`
- `Path path = Paths.get("uploads/" + filename);`
- `Resource resource = new UrlResource(path.toUri());`
- `return ResponseEntity.ok()`
- `.contentType(MediaType.APPLICATION_OCTET_STREAM)`
- `.header(HttpHeaders.CONTENT_DISPOSITION, "attachment;`
- `filename=\"\" + path.getFileName() + "\"\"")`
- `.body(resource);`
- `}`

Word to Pdf

- `@PostMapping("/word-to-pdf")`
- `public ResponseEntity<String>`
`convertDocToPdf(@RequestParam("file") MultipartFile file) throws`
`Exception {`
- `XWPFDocument doc = new`
`XWPFDocument(file.getInputStream());`
- `String outputPath = "uploads/converted.pdf";`
- `PdfOptions options = PdfOptions.create();`
- `OutputStream out = new FileOutputStream(outputPath);`
- `PdfConverter.getInstance().convert(doc, out, options);`
- `return ResponseEntity.ok("Converted Word to PDF at: " +`
`outputPath);`
- `}`

PDF to Word

- `@PostMapping("/pdf-to-word")`
- `public ResponseEntity<String> convertPdfToDocx(@RequestParam("file") MultipartFile file) throws IOException {`
- `String outputPath = "uploads/converted.docx";`
- `PDDocument document = PDDocument.load(file.getInputStream());`
- `PDFTextStripper stripper = new PDFTextStripper();`
- `String text = stripper.getText(document);`
- `XWPFDocument doc = new XWPFDocument();`
- `XWPFPParagraph para = doc.createParagraph();`
- `XWPFRun run = para.createRun();`
- `run.setText(text);`
- `FileOutputStream out = new FileOutputStream(outputPath);`
- `doc.write(out);`
- `out.close();`
- `document.close();`
- `return ResponseEntity.ok("Converted PDF to Word at: " + outputPath);`
- `}`

JPG to PDF

- `@PostMapping("/jpg-to-pdf")`
- `public ResponseEntity<String> convertJpgToPdf(@RequestParam("file") MultipartFile file) throws IOException {`
- `PDDocument doc = new PDDocument();`
- `BufferedImage image = ImageIO.read(file.getInputStream());`
- `PDPage page = new PDPage(new PDRectangle(image.getWidth(), image.getHeight()));`
- `doc.addPage(page);`
- `PDImageXObject pdImage = LosslessFactory.createFromImage(doc, image);`
- `PDPageContentStream content = new PDPageContentStream(doc, page);`
- `content.drawImage(pdImage, 0, 0);`
- `content.close();`
- `String outputPath = "uploads/image.pdf";`
- `doc.save(outputPath);`
- `doc.close();`
- `return ResponseEntity.ok("Converted JPG to PDF at: " + outputPath);`
- `}`

PDF to JPG

- `@PostMapping("/pdf-to-jpg")`
- `public ResponseEntity<String> convertPdfToJpg(@RequestParam("file") MultipartFile file) throws IOException {`
- `PDDocument document = PDDocument.load(file.getInputStream());`
- `PDFRenderer pdfRenderer = new PDFRenderer(document);`
- `String outputPath = "uploads/page.jpg";`
- `BufferedImage image = pdfRenderer.renderImageWithDPI(0, 300);`
- `ImageIO.write(image, "JPEG", new File(outputPath));`
- `document.close();`
- `return ResponseEntity.ok("Converted first page of PDF to JPG at: " + outputPath);`
- `}`

Simple UI with Drag and Drop

```
• <!-- index.html -->
• <!DOCTYPE html>
• <html>
• <head>
• <title>PDF Processor</title>
• <style>
•   #drop-zone {
•     width: 300px; height: 200px;
•     border: 2px dashed #ccc; border-radius: 10px;
•     text-align: center; line-height: 200px;
•     margin: 50px auto;
•   }
• </style>
• </head>
• <body>
• <div id="drop-zone">Drop files here</div>
• <input type="file" id="fileInput" style="display:none;" multiple>
• <script>
•   const dropZone = document.getElementById('drop-zone');
•   const fileInput = document.getElementById('fileInput');

•   dropZone.addEventListener('click', () => fileInput.click());
•   dropZone.addEventListener('dragover', e => {
•     e.preventDefault(); dropZone.style.background = '#eef';
•   });
•   dropZone.addEventListener('dragleave', () => dropZone.style.background = '#fff');
•   dropZone.addEventListener('drop', e => {
•     e.preventDefault();
•     const files = e.dataTransfer.files;
•     const formData = new FormData();
•     for (let file of files) formData.append("file", file);
•     fetch("/upload", { method: "POST", body: formData })
•       .then(res => res.text()).then(alert);
•   });
• </script>
• </body>
• </html>
```

Next

- <dependencies>
- <dependency>
- <groupId>org.springframework.boot</groupId>
- <artifactId>spring-boot-starter-web</artifactId>
- </dependency>
- <dependency>
- <groupId>org.apache.pdfbox</groupId>
- <artifactId>pdfbox</artifactId>
- <version>2.0.27</version>
- </dependency>
- <dependency>
- <groupId>com.itextpdf</groupId>
- <artifactId>itext7-core</artifactId>
- <version>7.1.15</version>
- <type>pom</type>
- </dependency>
- <dependency>
- <groupId>org.docx4j</groupId>
- <artifactId>docx4j</artifactId>
- <version>11.2.8</version>
- </dependency>
- <dependency>
- <groupId>commons-io</groupId>
- <artifactId>commons-io</artifactId>
- <version>2.8.0</version>
- </dependency>
- </dependencies>

JAVASCRIPT

- `let uploadedFiles = [];`
- `function upload() {`
- `const input = document.getElementById('pdfInput');`
- `const formData = new FormData();`
- `formData.append("file", input.files[0]);`
- `fetch("/api/upload", {`
- `method: "POST",`
- `body: formData`
- `}).then(res => res.text()).then(console.log);`
- `}`
- `function merge() {`
- `const input = document.getElementById('pdfInput');`
- `const formData = new FormData();`
- `for (let file of input.files) {`
- `formData.append("files", file);`
- `}`
- `fetch("/api/merge", {`
- `method: "POST",`
- `body: formData`
- `}).then(res => res.blob())`
- `.then(blob => {`
- `const url = window.URL.createObjectURL(blob);`
- `window.open(url);`
- `});`
- `}`
- `function split() {`
- `const input = document.getElementById('pdfInput');`
- `const formData = new FormData();`
- `formData.append("file", input.files[0]);`
- `formData.append("page", 0);`
- `fetch("/api/split", {`
- `method: "POST",`
- `body: formData`
- `}).then(res => res.blob())`
- `.then(blob => {`
- `const url = window.URL.createObjectURL(blob);`
- `window.open(url);`
- `});`
- `}`

FILE SERVICE.JAVA

```
• @Service
• public class FileService {

•     public String saveFile(MultipartFile file) {
•         try {
•             Path path = Paths.get("uploads/" + file.getOriginalFilename());
•             Files.copy(file.getInputStream(), path, StandardCopyOption.REPLACE_EXISTING);
•             return path.toString();
•         } catch (IOException e) {
•             throw new RuntimeException("Failed to save file", e);
•         }
•     }

•     public ByteArrayResource mergePDFs(MultipartFile[] files) {
•         try (ByteArrayOutputStream baos = new ByteArrayOutputStream()) {
•             PDFMergerUtility merger = new PDFMergerUtility();
•             merger.setDestinationStream(baos);

•             for (MultipartFile file : files) {
•                 merger.addSource(file.getInputStream());
•             }

•             merger.mergeDocuments(null);
•             return new ByteArrayResource(baos.toByteArray());

•         } catch (IOException e) {
•             throw new RuntimeException("Merge failed", e);
•         }
•     }

•     public ByteArrayResource splitPDF(MultipartFile file, int pageNumber) {
•         try (PDDocument document = PDDocument.load(file.getInputStream())) {
•             PDDocument newDoc = new PDDocument();
•             newDoc.addPage(document.getPage(pageNumber));
•             ByteArrayOutputStream baos = new ByteArrayOutputStream();
•             newDoc.save(baos);
•             newDoc.close();
•             return new ByteArrayResource(baos.toByteArray());
•         } catch (IOException e) {
•             throw new RuntimeException("Split failed", e);
•         }
•     }
• }
```

Next

- `@RestController`
- `@RequestMapping("/api/files")`
- `public class FileController {`
- `private final String uploadDir = "uploads/";`
- `@PostMapping("/upload")`
- `public ResponseEntity<?> uploadFiles(@RequestParam("files") MultipartFile[] files) {`
- `List<String> fileNames = new ArrayList<>();`
- `try {`
- `Files.createDirectories(Paths.get(uploadDir));`
- `for (MultipartFile file : files) {`
- `String filePath = uploadDir + file.getOriginalFilename();`
- `file.transferTo(Paths.get(filePath));`
- `fileNames.add(file.getOriginalFilename());`
- `}`
- `return ResponseEntity.ok("Uploaded files: " + String.join(", ", fileNames));`
- `} catch (IOException e) {`
- `return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body("File upload error: " +`
- `e.getMessage());`
- `}`
- `}`
- `}`

BACKEND

- `spring.datasource.url=jdbc:mysql://localhost:3306/pdf_app`
- `spring.datasource.username=root`
- `spring.datasource.password=yourpassword`
- `spring.jpa.hibernate.ddl-auto=update`
- `file.upload-dir=uploads/`

PdfAppApplication.JAVA

- @SpringBootApplication
- public class PdfAppApplication {
- public static void main(String[] args) {
- SpringApplication.run(PdfAppApplication.class
 , args);
- }
- }

DATABASE

- CREATE TABLE file_logs (
 - id INT AUTO_INCREMENT PRIMARY KEY,
 - filename VARCHAR(255),
 - action VARCHAR(50),
 - timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
-);

End.

•